The AI Stack

From infrastructure to application

A map of how AI systems are built — and where the money, the work, and the value actually are.

Agenda

1. **Why a stack? —** a model is one piece
2. **The 5 layers —** infra → models → data → orchestration → application
3. **The business view —** revenue pyramid · 3 types of innovation
4. **Case studies —** chatbot · RAG · the vendor-vs-operator trap
5. **AI in DevOps —** Copilot · AIOps · the DORA counterpoint
6. **Takeaways**

Why a "stack"?

A model is **one piece**, not the whole system.

To solve a real problem you also need: **compute** to run it, **data** to ground it, **orchestration** to coordinate it, and an **application** for the user.

Every choice across the stack drives **quality · speed · cost · safety**.

The 5 layers

```
5 · APPLICATION     interfaces & integrations — the user
4 · ORCHESTRATION   plan → execute → review — the "brain" (agentic)
3 · DATA            sources, pipelines, vector store, RAG
2 · MODELS          open/proprietary · size · specialization
1 · INFRASTRUCTURE  GPUs — on-premise, cloud, local
```

We'll go bottom → top, then look at the **business** picture.

1 · Infrastructure

LLMs need **AI-specific hardware (GPUs)**.

Three ways to deploy:

- **On-premise —** own it; full control, high upfront cost
- **Cloud —** rent it; scale up/down on demand
- **Local —** laptop; only the smaller models

> The hardware you can access decides what models you can run at all.

2 · Models

Pick along three dimensions:

- **Open vs proprietary —** control, cost, where it runs
- **Size —** large (LLM) vs small (SLM)
- **Specialization —** reasoning, tool-calling, code, language

Use vs build: consume a model, **fine-tune** it, or (rarely) **train** it.
Lifecycle glue = **MLOps**.

> Need fresh knowledge, not new behavior? Prefer RAG over fine-tuning.

3 · Data

Base models have a **knowledge cutoff** and don't know your private data.

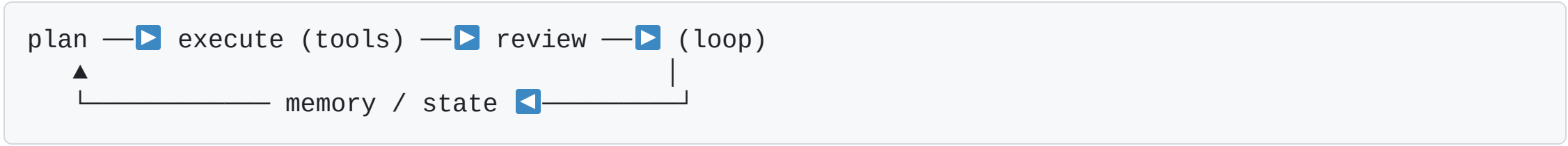
```
sources → pipelines (clean/chunk) → embed → vector store → RAG → model
```

RAG retrieves relevant context and augments the prompt — fresh & private knowledge without retraining.

4 · Orchestration — the agentic layer

One prompt in / one answer out = a **chat box**.

Add the loop and it becomes an **agent**:



Two kinds: **static** (reliable workflows) + **agentic** (autonomous decisions).

Fastest-moving layer — agents, MCP.

5 · Application

Where AI meets the user. Two questions:

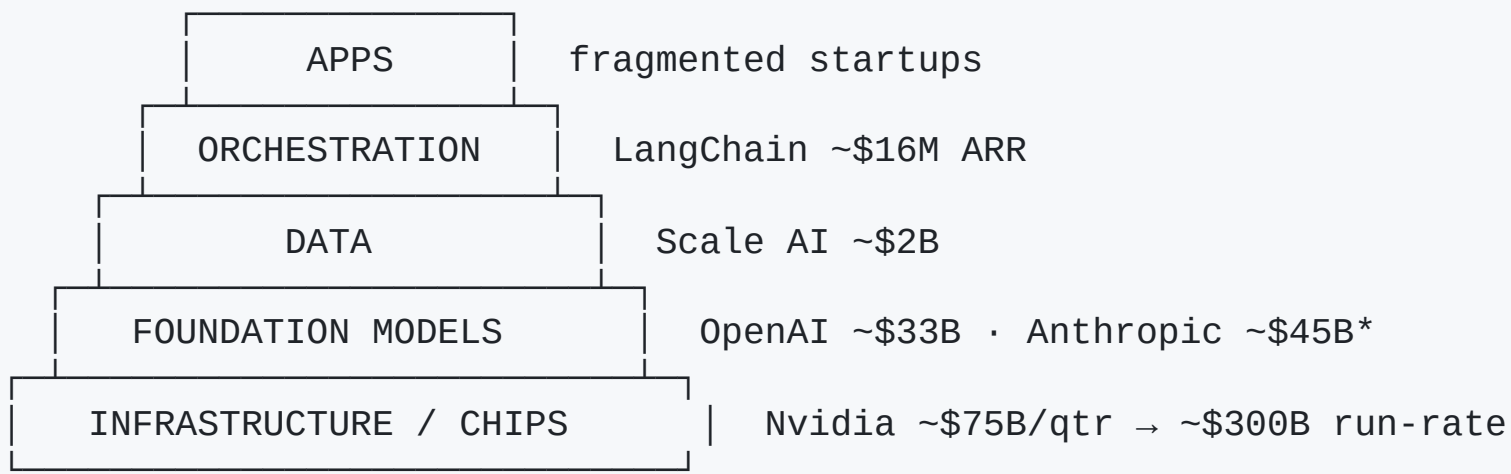
- **Interfaces —** text / image / audio · revisions · citations · *conversational UI replacing forms*
- **Integrations —** inbound (tools feed AI) & outbound (AI output → tools)

> "What the AI *does*" is orchestration. "What the user sees / where the result lands" is the application layer.

The business view

Where is the money — and where is the value?

Revenue pyramid (mid-2026)



Revenue concentrates at the **base** (Nvidia alone > all model companies).
Hype ≠ revenue — orchestration gets the most attention, the least money (LangChain ≈ 1/18,000 of Nvidia).

Three types of innovation

Clayton Christensen — a lens for any AI product

Type	Idea	Jobs	AI example
Market-creating	expensive/hard → cheap for all	creates	foundation models for everyone
Sustaining	make a good product better	neutral	an AI feature in your app
Efficiency	same work, less cost	reduces	coding assistants, automation

The type isn't the *layer* — it's how the tech is **used**.

Case · sales / support chatbot

Klarna × OpenAI: 2.3M chats in month 1 = **700 agents'** work,
67% automated, 11 min → **<2 min**, **~$40M** profit (2024).
Caveat: cut too deep, rehired in 2025 — AI-first, not AI-only.

```
ILLUSTRATIVE — 100,000 contacts/mo · human $5 · AI $1 · 67% deflection
  AI handles  67,000 × $1 = $67k/mo
  vs humans   67,000 × $5 = $335k/mo
  net saving  ≈ $268k/mo  ≈ $3.2M/yr
```

Players: Sierra · Decagon · Ada · Intercom Fin

Case · RAG over internal docs

Morgan Stanley × OpenAI

- Indexed **350,000 documents (40M words)** with RAG
- Before: **30+ min** manual search · advisors reached **~20%** of knowledge
- After: instant · **98%** of teams use it · access **20%** → **80%**

Frees advisor time for client work; answers grounded in verified sources.

Similar: Glean · Harvey · Hebbia

The trap: vendor value vs operator value

The pyramid measures who earns money **selling** AI → base wins.

Case studies measure value from **applying** AI → lands on the *operator's* books.

> Klarna's $40M isn't an "AI app company" revenue line — it's on **Klarna's** P&L. The app layer looks thin for vendors, yet is where operators win.

AI in DevOps

Speed is easy to measure. Value is not.

DevOps · coding assistant

GitHub Copilot (Accenture RCT): ~55% faster, +8.7% PRs,
+15% merge rate, +84% successful builds.

```
ILLUSTRATIVE — 100 devs · ~$120k/dev/yr
  freed capacity ≈ 30% × 55% ≈ 16% ≈ ~$1.9M/yr  (if realized)
  cost: Copilot $19/user/mo + tokens ≈ ~$30k/yr
  net ≈ ~$1.87M/yr
```

Revenue flat ⇒ this is **cost-cutting**. Subscription is cheap; the risk is **non-realization —** freed time becomes slack, not savings.

DevOps · AIOps (incident response)

PagerDuty: **–91% alert noise** · K8s MTTR **20 → <3 min** ·
HCL+Moogsoft: MTTR **–33%**, tickets **–62%**

```
ILLUSTRATIVE
SAVINGS  SRE time ~$12.8k/mo + downtime avoided ~$40k/mo ≈ $52.8k/mo
COST     license + tokens + setup + maintenance       ≈ $20–30k/mo
NET                                                    ≈ ~$28k/mo
```

A "–40% MTTR" headline says nothing about ROI until you **subtract the AI's own cost**.

DevOps · the counterpoint (DORA 2024)

- **75.9%** of devs use AI daily · ~75% feel more productive
- **But:** delivery **throughput −1.5%**, **stability −7.2%** as adoption rose
- Cause: batch sizes grow, trust in AI output rises

> **Individual speed ≠ delivery performance.** Without small batches, testing, and disciplined CI/CD, AI can make shipping *worse*. That gap is where DevOps creates value.

Takeaways

1. AI is a **stack**, not a model **—** infra → models → data → orchestration → app
2. Money sits at the **base**; **value** from *applying* AI sits with the **operator**
3. Name the innovation type: **market-creating / sustaining / efficiency**
4. Always do the math with **both sides —** savings *and* the AI's own cost
5. In DevOps, **fundamentals still win**

Thank you

Questions?

Speaker notes & full numbers: `README.md` *and* `docs/devops-cases.md`